

Assignment week 2

Q1. Write a query to display all columns and rows from the customer's table.

SELECT * FROM customers;

The screenshot shows a SQL IDE with a query editor and a result grid. The query editor contains the following SQL code:

```
104 * SELECT * FROM customers;
105 * SELECT * FROM products;
106 * SELECT * FROM orders;
107 * SELECT * FROM order_items;
108
109 * describe customers;
110
```

The result grid displays the following data:

customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0

Q2. Retrieve only the first_name, last_name, and city of all customers.

SELECT first_name, last_name, city
FROM customers;

The screenshot shows a SQL IDE with a query editor and a result grid. The query editor contains the following SQL code:

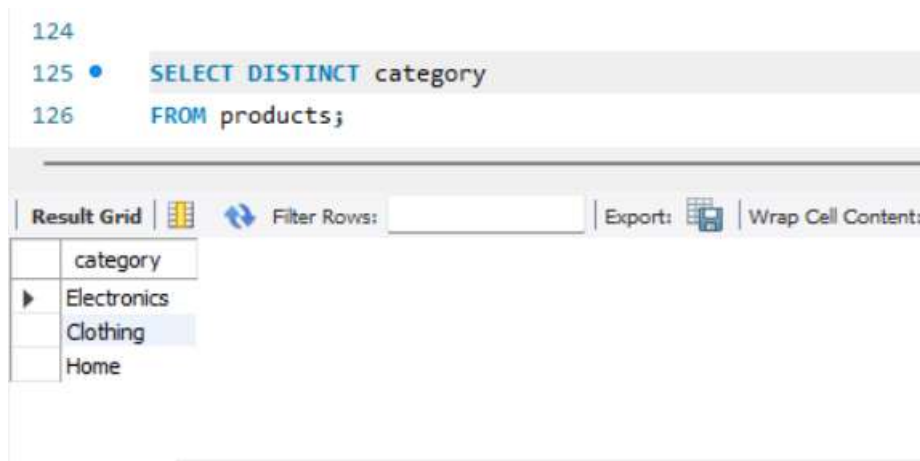
```
119
120 * SELECT first_name, last_name, city
121 FROM customers;
```

The result grid displays the following data:

first_name	last_name	city
Aarav	Sharma	Mumbai
Priya	Patel	Ahmedabad
Rohan	Gupta	Delhi
Sneha	Reddy	Hyderabad
Vikram	Singh	Jaipur

Q3. List all unique categories available in the products table.

SELECT DISTINCT category
FROM products;



Q4. Identify the Primary Key of each table in the schema. Explain why a Primary Key must be unique and NOT NULL.

Primary Keys:

customers -> customer_id

products -> product_id

orders -> order_id

order_items -> item_id

A Primary Key uniquely identifies each record.

It cannot contain NULL values and duplicate values.

Q5. What constraints are applied to the email column in the customers table? What would happen if you tried to insert a duplicate email?

email column has:

1. UNIQUE constraint
2. NOT NULL constraint

Duplicate email insertion will fail with a UNIQUE constraint error.

Q6. Try inserting a product with unit_price = -50. What happens and which constraint prevents it? Write both the INSERT statement and explain the error.

INSERT INTO products

(product_id, product_name, category, brand, unit_price, stock_qty)

VALUES

(209, 'Hanging clock', 'Electronics', 'Ajanta', '-50', 200);

The SQL statement failed because the value -50 was inserted into the unit_price column, which violates a **CHECK constraint** defined on the products table. A CHECK constraint is used to ensure that only valid data can be stored

in a column.

```
127
128 * INSERT INTO products
129 (product_id, product_name, category, brand, unit_price, stock_qty)
130 VALUES
131 (209, 'Hanging clock', 'Electronics', 'Ajanta', '-50', 200);
```

Output

#	Time	Action	Message
8	07:04:23	SELECT * FROM products LIMIT 0, 1000	8 row(s) returned
9	07:06:43	INSERT INTO products (product_id, product_name, category, brand, unit_price, stock_qty) VALUES (209, 'Ha...	Error Code: 3819. Check constraint 'products_chk_1' is violated.

Q7. Retrieve all orders with status = 'Delivered'.

```
SELECT *
FROM orders
WHERE status = 'Delivered';
```

```
132
133 * SELECT *
134 FROM orders
135 WHERE status = 'Delivered';
```

Result Grid

	order_id	customer_id	order_date	status	total_amount
▶	1001	101	2024-08-01	Delivered	4498.00
	1002	102	2024-08-03	Delivered	799.00
	1004	101	2024-08-10	Delivered	3499.00
	1006	105	2024-08-15	Delivered	5898.00
	1008	103	2024-08-20	Delivered	899.00
	1010	108	2024-08-28	Delivered	1598.00
	NULL	NULL	NULL	NULL	NULL

orders 8 x

Q8. Find all products in the 'Electronics' category with a unit_price greater than ₹2000.

```
SELECT * FROM PRODUCTS
WHERE CATEGORY = 'ELECTRONICS'
AND UNIT_PRICE > 2000;
```

136

```
137 • SELECT *  
138 FROM products  
139 WHERE category = 'Electronics'  
140 AND unit_price > 2000;
```

Result Grid						
Filter Rows:						
Edit:						
Export/Import:						
Wrap Cell Content:						
	product_id	product_name	category	brand	unit_price	stock_qty
▶	203	Smart Watch	Electronics	Noise	2999.00	150
	205	Bluetooth Speaker	Electronics	JBL	3499.00	200
•	NULL	NULL	NULL	NULL	NULL	NULL

products 11 x

Q9. List all customers who joined in the year 2024 and belong to the state 'Maharashtra'.

```
SELECT * FROM CUSTOMERS
```

```
WHERE YEAR(JOIN_DATE) = 2024
```

```
AND STATE = 'Maharashtra';
```

141

```
142 • SELECT * FROM customers  
143 WHERE YEAR(join_date) = 2024  
144 AND state = 'Maharashtra';
```

Result Grid								
Filter Rows:								
Edit:								
Export/Import:								
Wrap Cell Content:								
	customer_id	first_name	last_name	email	city	state	join_date	is_premium
▶	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
	107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
•	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Q10. Find all orders placed between '2024-08-10' and '2024-08-25' (inclusive) that are NOT cancelled.

```
SELECT *
```

```
FROM orders
```

```
WHERE order_date BETWEEN '2024-08-10' AND '2024-08-25'
```

```
AND status != 'cancelled';
```

```
145
146 • SELECT *
147 FROM orders
148 WHERE order_date BETWEEN '2024-08-10' AND '2024-08-25'
149 AND status != 'cancelled';
```

Result Grid | Filter Rows: | Edit: | Export/Import: |

order_id	customer_id	order_date	status	total_amount
1004	101	2024-08-10	Delivered	3499.00
1006	105	2024-08-15	Delivered	5898.00
1007	106	2024-08-18	Pending	1299.00
1008	103	2024-08-20	Delivered	899.00
1009	107	2024-08-25	Shipped	6098.00
HULL	HULL	HULL	HULL	HULL

orders 13 x

Q11. Explain what the index idx_orders_date does. How would it improve the performance of a query that filters orders by order_date? Write a sample query that would benefit from this index.

idx_orders_date is an index on order_date. It allows MySQL to locate matching dates quickly instead of scanning every row.

Example:

```
SELECT * FROM orders WHERE order_date='2024-08-15';
```

Q12. If you run: SELECT * FROM customers WHERE YEAR(join_date) = 2024; — would the index on join_date be used? Explain why or why not, and rewrite the query to be index-friendly (SARGable).

Explanation: No, the index would not be used. Because you wrapped the column in the YEAR() function, the database is forced to evaluate that function on every single row before it can check the condition. This breaks the index usage (making it non-SARGable) and causes a slow full table scan.

```
1 • SELECT *
2 FROM customers
3 WHERE join_date >= '2024-01-01'
4 AND join_date < '2025-01-01';
```

ult Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: |

customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
107	Karan	Mukherjee	karan.m@email.com	Pune	Maharashtra	2024-07-22	1

omers 14 x

Q13. Count the total number of orders in the orders table

```
SELECT COUNT(*) AS total_orders
```

```
FROM orders;
```

```
156 • SELECT COUNT(*) AS total_orders
157 FROM orders;
```

Result Grid	Filter Rows:	Export:
total_orders		
10		

Q14. Find the total revenue from all Delivered orders

```
SELECT SUM(total_amount) AS total_revenue
```

```
FROM orders
```

```
WHERE status = 'Delivered';
```

```
158
159 • SELECT SUM(total_amount) AS total_revenue
160 FROM orders
161 WHERE status = 'Delivered';
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
total_revenue			
17191.00			

Q15. Calculate the average unit_price of products in each category

```
SELECT category, AVG(unit_price) AS avg_uunit_price
```

```
FROM products
```

```
GROUP BY category;
```

```

163
164 • SELECT category, AVG(unit_price) AS avg_uunit_price
165 FROM products
166 GROUP BY category;

```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
category	avg_uunit_price			
Electronics	2224.000000			
Clothing	2699.000000			
Home	949.000000			

Q16. For each order status, find the count of orders and total revenue. Sort by revenue descending

```

SELECT status, COUNT(*) AS order_count, SUM(total_amount) AS total_revenue
FROM orders
GROUP BY status
ORDER BY total_revenue DESC;

```

```

168 • SELECT status, COUNT(*) AS order_count, SUM(total_amount) AS total_revenue
169 FROM orders
170 GROUP BY status
171 ORDER BY total_revenue DESC;

```

Result Grid				Filter Rows:	Export:	Wrap Cell Content:
	status	order_count	total_revenue			
▶	Delivered	6	17191.00			
	Shipped	2	13596.00			
	Cancelled	1	2999.00			
	Pending	1	1299.00			

Q17. Find the most expensive and cheapest product in each category

```

SELECT category, MAX(unit_price) AS most_expensive, MIN(unit_price) AS cheapest
FROM products
GROUP BY category;

```



```

174
175 • SELECT category, MAX(unit_price) AS most_expensive, MIN(unit_price) AS cheapest
176 FROM products
177 GROUP BY category;
178

```

category	most_expensive	cheapest
Electronics	3499.00	899.00
Clothing	4599.00	799.00
Home	1299.00	599.00

Q18. List all product categories where the average unit_price is greater than ₹2000

```

SELECT category,
        AVG(unit_price) AS avg_unit_price
FROM products
GROUP BY category
HAVING AVG(unit_price) > 2000;

```

```

.78
.79 • SELECT category,
.80         AVG(unit_price) AS avg_unit_price
.81 FROM products
.82 GROUP BY category
.83 HAVING AVG(unit_price) > 2000;
.84

```

category	avg_unit_price
Electronics	2224.000000
Clothing	2699.000000

Q19. INNER JOIN: Display order details with customer name

```

SELECT o.order_id, o.order_date, c.first_name, c.last_name, o.total_amount
FROM orders o
INNER JOIN customers c

```


ON o.customer_id = c.customer_id;

Explanation

- INNER JOIN returns only matching records from both tables.
- Here, only orders that have a valid customer will be displayed.

```
184
185 ✖ SELECT o.order_id,
186         o.order_date,
187         c.first_name,
188         c.last_name,
189         o.total_amount
190 FROM orders o
191 INNER JOIN customers c
192 ON o.customer_id = c.customer_id;
193
```

order_id	order_date	first_name	last_name	total_amount
1001	2024-08-01	Aarav	Sharma	4498.00
1004	2024-08-10	Aarav	Sharma	3499.00
1002	2024-08-03	Priya	Patel	799.00
1003	2024-08-05	Rohan	Gupta	7498.00
1008	2024-08-20	Rohan	Gupta	899.00
1005	2024-08-12	Sneha	Reddy	2999.00
1006	2024-08-15	Vikram	Singh	5000.00

Q20. LEFT JOIN: List all customers and their orders

SELECT c.customer_id, c.first_name, c.last_name, o.order_id, o.order_date, o.status, o.total_amount

FROM customers c

LEFT JOIN orders o ON c.customer_id = o.customer_id;

Explanation

- LEFT JOIN returns all customers.
- If a customer has no orders, order-related columns will show NULL.

```

93
94 • SELECT c.customer_id,
95       c.first_name,
96       c.last_name,
97       o.order_id,
98       o.order_date,
99       o.status,
100      o.total_amount
101 FROM customers c
102 LEFT JOIN orders o
103 ON c.customer_id = o.customer_id;
104

```

customer_id	first_name	last_name	order_id	order_date	status	total_amount
101	Aarav	Sharma	1001	2024-08-01	Delivered	4498.00
101	Aarav	Sharma	1004	2024-08-10	Delivered	3499.00
102	Priya	Patel	1002	2024-08-03	Delivered	799.00
103	Rohan	Gupta	1003	2024-08-05	Shipped	7498.00
103	Rohan	Gupta	1008	2024-08-20	Delivered	899.00
104	Sneha	Reddy	1005	2024-08-12	Cancelled	2999.00
105	Vijay	Clark	1006	2024-08-15	Delivered	5000.00

Q21. JOIN across three tables

Explanation

Relationship used:

orders

|

order_id

|

order_items

|

product_id

|

products

This query displays each order item along with the product details.

04

```
05 • SELECT o.order_id,  
06         p.product_name,  
07         oi.quantity,  
08         oi.unit_price,  
09         oi.discount_pct  
10 FROM orders o  
11 JOIN order_items oi  
12 ON o.order_id = oi.order_id  
13 JOIN products p  
14 ON oi.product_id = p.product_id;
```

15

order_id	product_name	quantity	unit_price	discount_pct
1001	Wireless Earbuds	2	1499.00	0.00
1001	Laptop Stand	1	899.00	10.00
1002	Cotton T-Shirt	1	799.00	0.00
1003	Smart Watch	1	2999.00	0.00
1003	Running Shoes	1	4599.00	5.00
1004	Bluetooth Speaker	1	3499.00	0.00
1005	Smart Watch	1	2999.00	0.00

Q22. Difference between LEFT JOIN and RIGHT JOIN

LEFT JOIN Returns:

- All rows from the left table
- Matching rows from the right table
- NULL when no match exists

Example:

```
SELECT * FROM customers c
```

```
LEFT JOIN orders o
```

```
ON c.customer_id = o.customer_id;
```

Result:

- All customers appear.
- Customers without orders show NULL for order columns.

RIGHT JOIN Returns:

- All rows from the right table
- Matching rows from the left table
- NULL when no match exists

Example:

```
SELECT * FROM customers c
```

```
RIGHT JOIN orders o
```

```
ON c.customer_id = o.customer_id;
```

Result:

- All orders appear.
- If an order had no matching customer, customer columns would be NULL.

FULL OUTER JOIN Returns:

- All rows from both tables
- Matching rows where available
- NULL where no match exists

Example use case:

- You want a complete list of all customers and all orders, including unmatched records from both sides.

Q23. Foreign Key Relationships

Orders → Customers

```
FOREIGN KEY (customer_id)
```

```
REFERENCES customers(customer_id)
```

Meaning: • Every order must belong to an existing customer.

Order_Items → Orders

```
FOREIGN KEY (order_id)
```

```
REFERENCES orders(order_id)
```

Meaning:

- Every order item must belong to an existing order.

Order_Items → Products

```
FOREIGN KEY (product_id)
```

```
REFERENCES products(product_id)
```

Meaning:

- Every order item must reference an existing product.

What happens if we insert an order with customer_id = 999?

```
INSERT INTO orders VALUES ( 1011, 999, '2024-09-01', 'Pending', 1000.00 );
```

Result MySQL will throw a Foreign Key Constraint Error.

Q24. Classify products into price tiers using CASE

```
SELECT product_name,  
       unit_price,  
       CASE  
         WHEN unit_price < 1000 THEN 'Budget'  
         WHEN unit_price BETWEEN 1000 AND 3000 THEN 'Mid-Range'  
         ELSE 'Premium'  
       END AS price_tier  
FROM products;
```

```
215  
216 • SELECT product_name,  
217       unit_price,  
218       CASE  
219         WHEN unit_price < 1000 THEN 'Budget'  
220         WHEN unit_price BETWEEN 1000 AND 3000 THEN 'Mid-Range'  
221         ELSE 'Premium'  
222       END AS price_tier  
223 FROM products;  
224
```

Result Grid			
Filter Rows:			
Export:  Wrap Cell Content: 			
product_name	unit_price	price_tier	
Wireless Earbuds	1499.00	Mid-Range	
Cotton T-Shirt	799.00	Budget	
Smart Watch	2999.00	Mid-Range	
Running Shoes	4599.00	Premium	
Bluetooth Speaker	3499.00	Premium	
Bedsheet Set	1299.00	Mid-Range	
Leather Stand	899.00	Budget	

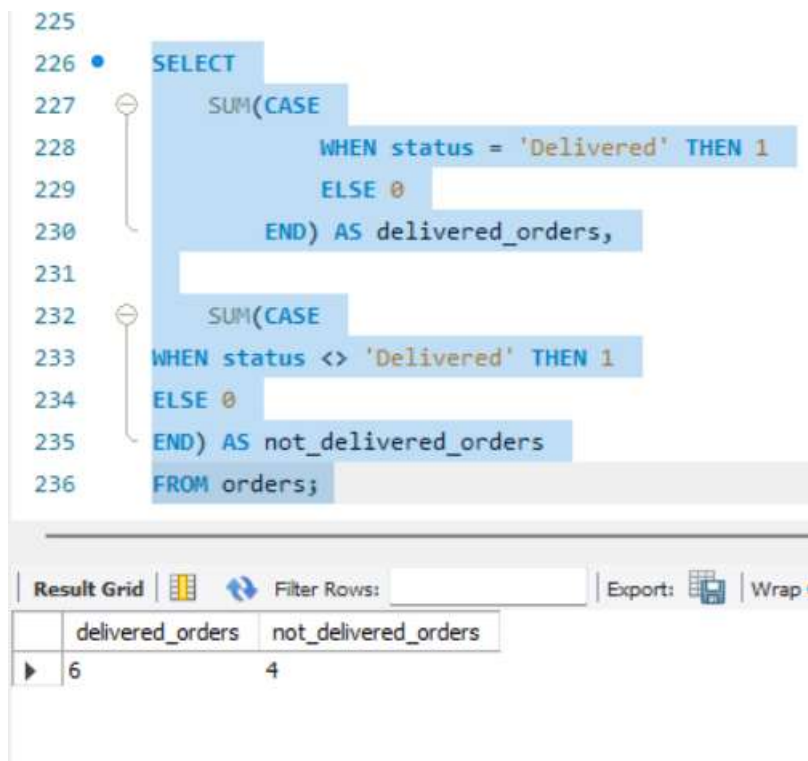
Q25. Count Delivered vs Not Delivered Orders

```
SELECT  
  
SUM(CASE  
  WHEN status = 'Delivered' THEN 1  
  ELSE 0  
END) AS delivered_orders,
```

```

SUM(CASE
WHEN status <> 'Delivered' THEN 1
ELSE 0
END) AS not_delivered_orders
FROM orders;

```



Explanation

For every row:

- Delivered → count as 1 in delivered_orders
- Other statuses (Pending, Shipped, Cancelled) → count as 1 in not_delivered_orders

Output will be a single row.

Q26. ACID Properties

A – Atomicity

Definition: A transaction is treated as a single unit. Either all operations succeed or none of them do.

Example (Bank Transfer) Suppose ₹1000 is transferred from Account A to Account B.

Steps:

1. Deduct ₹1000 from A
2. Add ₹1000 to B If the system crashes after Step 1 but before Step 2:

Without Atomicity:

- A loses money

- B doesn't receive money

With Atomicity:

- Entire transaction is rolled back.

C – Consistency

Definition: A transaction must move the database from one valid state to another valid state.

Example

Before transfer:

A = ₹5000

B = ₹3000

Total = ₹8000

After transfer:

A = ₹4000

B = ₹4000

Total = ₹8000

Consistency ensures database rules are never violated.

I – Isolation

Definition:

Multiple transactions should not interfere with each other.

Example Two users try to withdraw money from the same account simultaneously.

Without Isolation:

- Both transactions may read the same balance.
- Incorrect results occur.

Isolation ensures each transaction behaves as if it is running alone.

D – Durability

Definition:

Once a transaction is committed, it remains permanently stored.

Example After a bank transfer is successfully committed:

COMMIT; Even if:

- Power fails
- Server crashes

The data remains saved.

Q27. SQL Transaction

-- Step 1: Insert new order

INSERT INTO orders

(order_id, customer_id, order_date, status, total_amount)

VALUES

(1011, 102, CURDATE(), 'Pending', 1598.00);

-- Step 2: Insert order items

INSERT INTO order_items

(item_id, order_id, product_id, quantity, unit_price, discount_pct)

VALUES

(5016, 1011, 206, 1, 1299.00, 0);

INSERT INTO order_items

(item_id, order_id, product_id, quantity, unit_price, discount_pct)

VALUES

(5017, 1011, 208, 1, 599.00, 0);

-- Step 3: Update stock quantities

UPDATE products

SET stock_qty = stock_qty - 1

WHERE product_id = 206;

UPDATE products

SET stock_qty = stock_qty - 1

WHERE product_id = 208;

-- Step 4: Commit transaction

COMMIT;

Rollback Example

If any statement fails:

ROLLBACK;

Explanation

The transaction guarantees:

- Order insertion succeeds.
- Order items are inserted.

- Stock is updated.
- If any step fails, all previous changes are undone.

This demonstrates Atomicity, one of the ACID properties.

```
248 • INSERT INTO order_items
249     (item_id, order_id, product_id, quantity, unit_price, discount_pct)
250     VALUES
251     (5017, 1011, 208, 1, 599.00, 0);
252
253 • UPDATE products
254     SET stock_qty = stock_qty - 1
255     WHERE product_id = 206;
256
257 • UPDATE products
258     SET stock_qty = stock_qty - 1
259     WHERE product_id = 208;
260 • COMMIT;
261
262 • select * from order_items;
```

Result Grid

item_id	order_id	product_id	quantity	unit_price	discount_pct
5013	1009	208	2	599.00	15.00
5014	1010	206	1	1299.00	0.00
5015	1010	208	1	599.00	0.00
5016	1011	206	1	1299.00	0.00
5017	1011	208	1	599.00	0.00
NULL	NULL	NULL	NULL	NULL	NULL